ggvmap: Voronoi map treemaps for ggplot2 in pure R

Loukas Theodosiou

loukesio@gmail.com

Abstract Treemaps communicate part-of-whole relationships by mapping quantities to areas. Voronoi treemaps generalise the familiar rectangular layout to arbitrary convex boundaries and organic, polygon-shaped cells, but the existing R implementations delegate the layout either to a JavaScript/D3 runtime or to compiled CGAL code, which complicates installation, reproducibility, and integration with the ggplot2 ecosystem. Here I present ggvmap, an R package that computes Voronoi map treemaps in pure R – its only hard dependency is ggplot2 – and renders them as ordinary ggplot2 objects. ggvmap supports hierarchical (grouped) layouts, a grammar of pipe-able annotation layers (rings, flags, images, value labels), automatic small-cell label handling, and interactive hover maps. The tessellations it produces are exact power diagrams whose defining geometric properties are enforced by property-based tests. Benchmarks against voronoiTreemap (D3) and WeightedTreemaps (CGAL) show that ggvmap achieves comparable layout accuracy with a trivially simple installation, at the cost of longer runtimes for very large maps.

Highlights

- Voronoi map treemaps as native ggplot2 layers, in pure R with a single hard dependency (ggplot2).
- Hierarchical (grouped) layouts plus a grammar of pipe-able annotation verbs: rings (band or labelled arc), country flags, images, and value labels.
- Automatic small-cell handling: per-cell label autoscaling and area-based label suppression.
- Verified correctness: the layout is an exact power diagram, enforced by property-based tests on every build.

1 Introduction

Part-of-whole structure is among the most common shapes of scientific data: gene families within a genome, reads within a sequencing library, budget positions within an economy. Treemaps display such data by making the area of a cell proportional to its weight. The classic rectangular treemap is easy to compute but produces high-aspect-ratio slivers and a strongly grid-like appearance; *Voronoi treemaps* [1] instead partition an arbitrary convex polygon into organic cells, improving both aesthetics and area perception, and are the layout behind many widely shared data-journalism infographics.

Computing a Voronoi treemap requires iteratively adapting an additively weighted Voronoi diagram – a power diagram [2] – until every cell's area matches its target share [3]. In R, two implementations exist: **voronoiTreemap**, which wraps the D3/JavaScript layout in an htmlwidget [4], and **WeightedTreemaps**, which computes the layout in compiled C++ against the CGAL geometry library [5]. Both work well, but neither produces a ggplot2 object, and both carry installation burdens – a browser runtime in one case, CGAL compilation in the other – that complicate scripted, reproducible pipelines. There is no pure-R, ggplot2-native option. **ggvmap** fills that gap, following the design precedent of tidyplots [6] in offering a small, consistent, pipe-able interface on top of ggplot2 [7].

2 Results

2.1 A grammar of Voronoi maps

Figure 1 summarises the interface. A single computation function, `voronoi_map()`, takes weights (optionally with labels and a grouping vector) and returns the tessellation; `ggvmap()` renders it as a ggplot; and a family of `vm_add_*()` verbs – `vm_add_ring()`, `vm_add_labels()`, `vm_add_flags()`, `vm_add_images()` – add annotation layers through the native pipe, each reading the layout from the plot object and returning a new plot, so the layers

chain without intermediate variables (Figure 1 A, C). `ggvmap()` also accepts raw weights directly, collapsing computation and rendering into one call.

Appearance is controlled at three levels (Figure 1 D): the map (palette, fill mapping, clip shape, font family), the group (per-group border colours for emphasis), and the cell (per-cell font faces, label colours, and label selection). Arguments accept either a single value or a vector named by cell or group, so emphasising one region of an otherwise uniform map is a one-argument change. Because every result is a ggplot, titles, themes, and extensions from the wider ggplot2 ecosystem (e.g. markdown titles via ggtext, custom fonts via showtext) apply unchanged.

Figure 1: Design of ggvmap. (A) The workflow: one computation function, one plot function, pipe-able annotation verbs, and either static (`ggsave()`) or interactive (`vm_girafe()`) output. (B) From data to plot: the bundled `freshwater` dataset (share of global renewable freshwater resources, 2022 [8]) rendered by the code shown. (C) The annotation-layer verbs. (D) The three levels of appearance control.

2.2 One dataset, many maps

Figure 2 shows the same dataset across the package's layout and annotation space: nine convex boundary shapes, a hierarchical layout with one sector per region and per-group border emphasis, the two ring styles (filled band and labelled arc with group shares), continuous fill by weight, and flag annotations. Labels in small cells are automatically shrunk (`autoscale`) or suppressed (`min_area`), which keeps maps with long-tailed weight distributions legible without manual tuning.

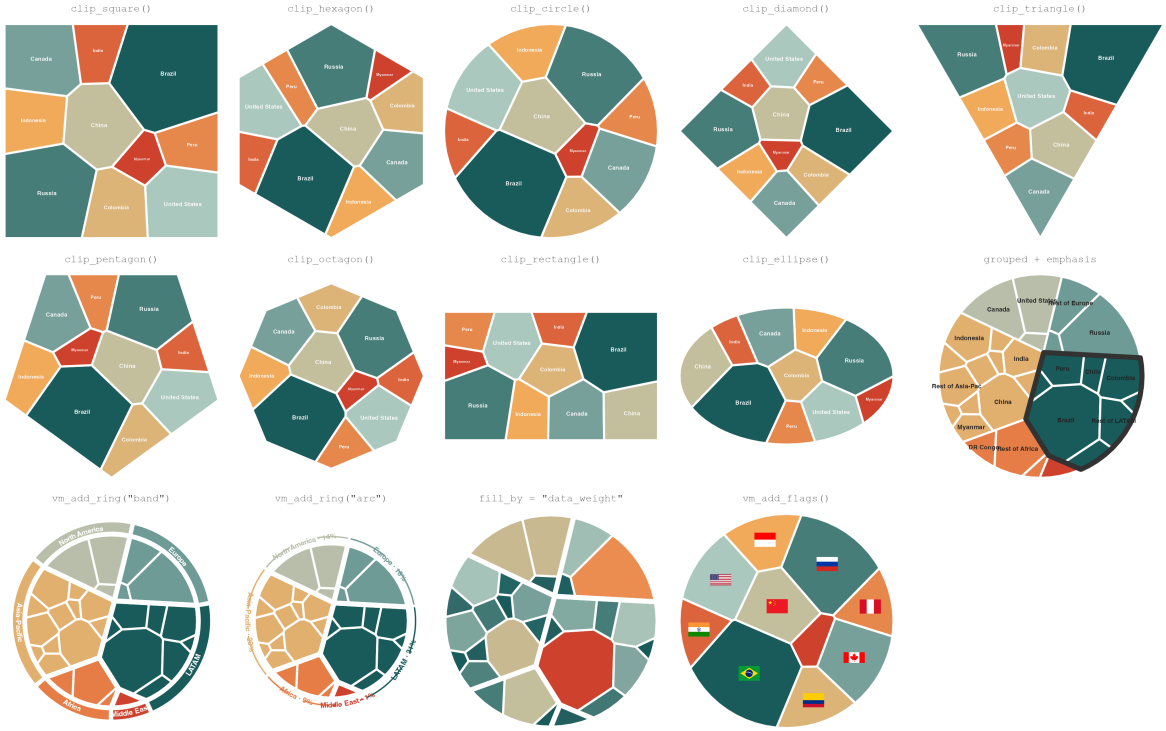

Figure 2: Gallery. All panels are drawn from `data(freshwater)`: the nine built-in convex boundaries (rows 1–2), and the grouped layout, ring styles, continuous fill, and flags (row 3).

2.3 Use cases

Figure 3 sketches four disciplinary applications, each fully scripted in the repository: gene families by genome share, genome-wide association hits per chromosome, RNA-seq library composition (with the contaminant fraction outlined for emphasis), and household expenditure shares. The data in this figure are simulated but realistic; the scripts are drop-in templates for real data.

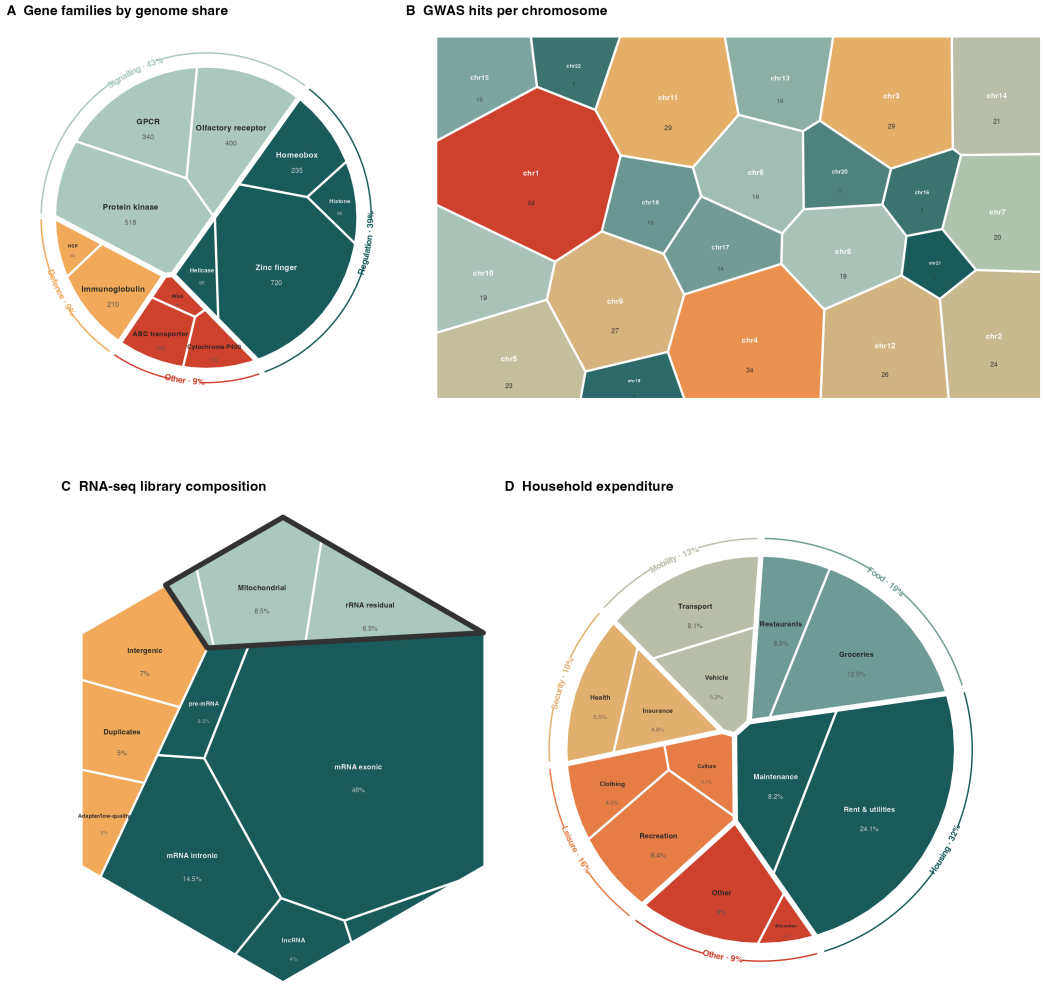

Figure 3: Use cases (simulated data; scripts in `paper/figures/`). (A) Gene families grouped by functional class. (B) GWAS hits per chromosome, continuous fill. (C) RNA-seq library composition with the contaminant group outlined. (D) Household expenditure by need category.

2.4 Benchmarks

Figure 4 compares ggvmap with voronoiTreemap and WeightedTreemaps on four axes. *Install complexity*: ggvmap needs 17 recursively resolved CRAN packages and no compiled code, versus 36 packages plus CGAL compilation for WeightedTreemaps; voronoiTreemap installs many htmlwidgets dependencies and runs its layout in a browser. *Code length*: the same grouped treemap takes the fewest words, characters, and function calls in ggvmap. *Runtime*: the picture is size-dependent. At small maps ggvmap is the faster of the two measurable implementations – 68 ms versus 664 ms at n = 10 and 2.5 s versus 4.5 s at n = 50 – because it avoids the fixed setup cost of the compiled pipeline. Beyond that, pure R pays its price: at n = 100 ggvmap needs 33.4 s versus 5.5 s, and at n = 500 the gap is 30-fold (14.5 min vs 29.5 s). Treemaps typically summarise tens of categories, where ggvmap is competitive or faster; for maps of many hundreds or thousands of cells, WeightedTreemaps is the right tool. *Accuracy*: on these deliberately long-tailed lognormal weights, both implementations concentrate their error in the very smallest cells, whose target areas are near zero – a large *relative* error there is visually invisible. Mean relative cell-area error for ggvmap ranges from 2% to 57% across sizes, against 5% to 42% for WeightedTreemaps; ggvmap's error is the lower of the two at three of the four sizes. voronoiTreemap computes its layout in the browser, so R-side runtime and accuracy are not measurable and it appears only in the first two panels.

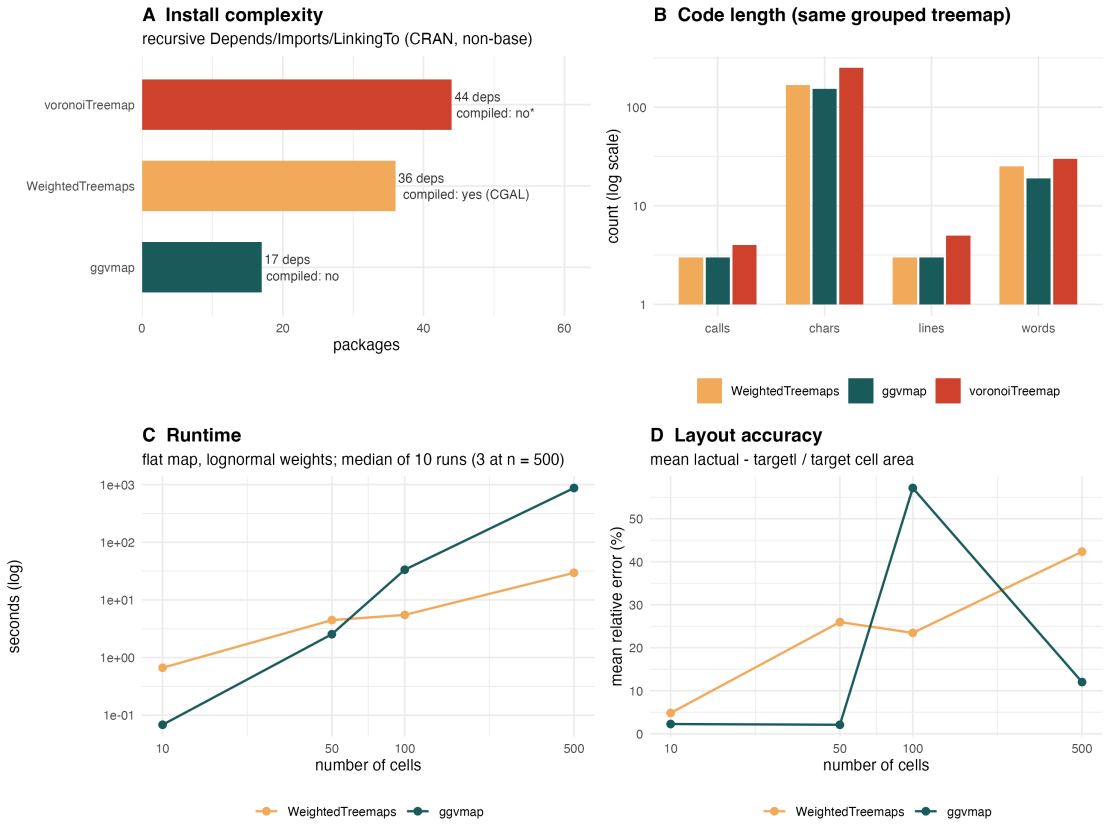

Figure 4: Benchmarks. (A) Recursive dependency count and compiled code. (B) Code length for the same grouped treemap. (C) Runtime on flat maps of lognormal weights (median of 10 runs; 3 at n = 500). (D) Mean relative cell-area error. Panels C–D omit voronoiTreemap, whose layout runs in the browser.

3 Conclusion

ggvmap makes Voronoi map treemaps a first-class citizen of the ggplot2 ecosystem: a one-line install, a small pipeable grammar, verified geometry, and publication-style annotation layers. The pure-R implementation trades raw speed at large cell counts for zero installation friction and full composability – a trade that favours the typical treemap, which summarises tens of categories, not thousands.

4 Methods

4.1 Interface design

The interface follows three rules, adopting the design vocabulary of tidyplots [6] and ggplot2 [7]: (i) one verb per action – `voronoi_map()` computes, `ggvmap()` renders, `vm_add_*()` annotate; (ii) every function both accepts and returns the natural object of its stage (weights to layout to plot), so calls compose with the pipe; (iii) arguments degrade gracefully from scalars to named vectors, so emphasis is expressed by naming cells or groups rather than by restructuring data. Annotation layers re-attach the layout object to the returned plot, which is what makes the chain stateless for the user.

4.2 Algorithm

The layout follows [3]. Each cell is a site with a position and an additive power weight; the plane is partitioned by the power diagram [2], computed here in pure R by sequential half-plane clipping of the convex boundary. Each iteration moves every site to its cell's centroid (position adaptation) and multiplicatively adjusts its power weight by the ratio of target to actual area (weight adaptation), until the maximum relative area error falls below a convergence threshold. Intuitively, the algorithm is a room full of balloons: each balloon drifts toward the centre of its territory and inflates or deflates until it holds exactly its share of the room. Hierarchical maps solve the group-level map first, then run the same algorithm inside each group polygon – a valid nested power diagram. Weights are initialised proportional to target areas and sites by farthest-point sampling, which removes degenerate starts.

4.3 Correctness

The test suite enforces the defining properties of the output rather than snapshots: every point of a fine grid lies in the cell of its minimum-power-distance site; cell areas sum to the clip area to machine precision with no gaps or overlaps; every cell is convex; and each hierarchical subcell lies inside its group polygon (`tests/testthat/test-correctness.R`). These property-based tests run on every build.

4.4 Dependencies

ggvmap Imports only ggplot2 (plus the base packages grDevices, stats, utils). Optional features degrade gracefully behind `requireNamespace()` guards: ggimage (flags, images), ggiraph (interactivity), geomtextpath (curved ring labels), magick and png (image caching).

4.5 Benchmarking

Benchmarks (Figure 4) were run with bench [9]: flat maps of n = 10, 50, 100, 500 lognormal(3, 1) weights, median of 10 iterations (3 at n = 500), on a single laptop core, each package at its default convergence settings. Dependency counts are recursive Depends, Imports and LinkingTo on CRAN, excluding base packages. Code-length metrics count lines, words, characters, and function calls of the minimal code producing the same grouped treemap in each package. Layout accuracy is the mean over cells of |actual - target| / target area, taken from each package's reported per-cell areas. The full script is `paper/figures/fig4_benchmark.R`; results are archived in `paper/figures/benchmark_results.csv`.

4.6 Data availability

ggvmap is available at https://github.com/loukesio/ggvmap under the MIT licence. The `freshwater` dataset ships with the package (FAO Aquastat via World Bank, 2022 [8]). All figures in this paper are generated by the scripts in `paper/figures/` and rendered by `paper/render.R`.

References

Bibliography

[1] M. Balzer and O. Deussen, "Voronoi Treemaps", in *Proceedings of the IEEE Symposium on Information Visualization (InfoVis)*, 2005, pp. 49–56. doi: 10.1109/INFVIS.2005.1532128.

[2] F. Aurenhammer, "Power Diagrams: Properties, Algorithms and Applications", *SIAM Journal on Computing*, vol. 16, no. 1, pp. 78–96, 1987, doi: 10.1137/0216006.

[3] A. Nocaj and U. Brandes, "Computing Voronoi Treemaps: Faster, Simpler, and Resolution-independent", *Computer Graphics Forum*, vol. 31, no. 3, pp. 855–864, 2012, doi: 10.1111/j.1467-8659.2012.03078.x.

[4] M. Li, E. Mills, and UOB HDRUK, "voronoiTreemap: Voronoi Treemaps with Added Interactivity by Shiny". [Online]. Available: https://cran.r-project.org/package=voronoiTreemap

[5] M. Jahn and Y. Leifels, "WeightedTreemaps: Generate and Plot Voronoi or Sunburst Treemaps from Hierarchical Data". [Online]. Available: https://cran.r-project.org/package=WeightedTreemaps

[6] J. B. Engler, "Tidyplots Empowers Life Scientists With Easy Code-Based Data Visualization", *iMeta*, vol. 4, p. e70018, 2025, doi: 10.1002/imt2.70018.

[7] H. Wickham, *ggplot2: Elegant Graphics for Data Analysis*. Springer-Verlag New York, 2016. [Online]. Available: https://ggplot2.tidyverse.org/

[8] Food and Agriculture Organization of the United Nations, "AQUASTAT: FAO's Global Information System on Water and Agriculture -- Renewable Internal Freshwater Resources". [Online]. Available: https://data.worldbank.org/indicator/ER.H2O.INTR.K3

[9] J. Hester and D. Vaughan, "bench: High Precision Timing of R Expressions". [Online]. Available: https://cran.r-project.org/package=bench